

Contents

1	Salesforce CRM Analytics Weekly Case Count Trending Report - Implementation Guide	1
1.1	Overview	1
1.2	Prerequisites	1
1.3	Implementation Steps	1
1.3.1	Step 1: Data Preparation and Dataset Creation	1
1.3.2	Step 2: Create the Weekly Trending Query	2
1.3.3	Step 3: Create Dashboard Components	3
1.3.4	Step 4: Add Interactive Filters	4
1.3.5	Step 5: Advanced Features	4
1.3.6	Step 6: Dashboard Layout and Design	5
1.3.7	Step 7: Testing and Validation	6
1.3.8	Step 8: Deployment and Sharing	6
1.4	Best Practices	6
1.4.1	Performance Optimization	6
1.4.2	Data Quality	6
1.4.3	User Experience	7
1.5	Troubleshooting Common Issues	7
1.5.1	Query Performance Issues	7
1.5.2	Date Handling Problems	7
1.5.3	Filter Not Working	7
1.5.4	Dashboard Not Loading	7
1.6	Maintenance and Updates	7
1.6.1	Regular Tasks	7
1.6.2	Version Control	7
1.7	Additional Resources	7
1.7.1	Salesforce Documentation	7
1.7.2	Training Resources	8

1 Salesforce CRM Analytics Weekly Case Count Trending Report - Implementation Guide

1.1 Overview

This guide provides detailed steps to build a trending report showing weekly case counts in Salesforce CRM Analytics dashboard. The report will display case volume trends over time with filtering capabilities and interactive visualizations.

1.2 Prerequisites

- Salesforce CRM Analytics (formerly Einstein Analytics) license
- Access to Salesforce org with Case data
- CRM Analytics Studio permissions
- Basic understanding of SAQL (Salesforce Analytics Query Language)

1.3 Implementation Steps

1.3.1 Step 1: Data Preparation and Dataset Creation

```
-- Check available Case fields in your org
SELECT Id, CaseNumber, CreatedDate, Status, Priority, Origin, Type, Subject, OwnerId, AccountId
FROM Case
LIMIT 10
```

1.3.1.1 1.1 Verify Case Data Access

1.3.1.2 1.2 Create or Verify Existing Dataset

- Navigate to **Analytics Studio** → **Data Manager**
- Look for existing Case dataset or create new one:
 - Click **Create** → **Dataset**
 - Select **Salesforce** as data source
 - Choose **Case** object
 - Select required fields:
 - * Id (Case ID)
 - * CaseNumber
 - * CreatedDate
 - * Status
 - * Priority
 - * Origin
 - * Type
 - * Subject
 - * OwnerId
 - * AccountId
 - * Account.Name (if needed)
 - * Owner.Name

1.3.1.3 1.3 Configure Dataset Settings

- Set **CreatedDate** as the primary date field
- Enable **Time-based data** if you want automatic refreshes
- Schedule data refresh (daily recommended)

1.3.2 Step 2: Create the Weekly Trending Query

```
q = load "YourCaseDatasetName";

-- Filter data for relevant time period (e.g., last 12 months)
q = filter q by 'CreatedDate_Year' >= 2023;

-- Group by week and count cases
q = group q by ('CreatedDate_Year', 'CreatedDate_Week');
q = foreach q generate
    'CreatedDate_Year' as 'Year',
    'CreatedDate_Week' as 'Week',
    count() as 'CaseCount',
    'CreatedDate_Year' + "-W" + 'CreatedDate_Week' as 'YearWeek';

-- Sort by year and week
q = order q by ('Year' asc, 'Week' asc);
```

1.3.2.1 2.1 Base SAQL Query for Weekly Case Counts

```
q = load "YourCaseDatasetName";

-- Filter for relevant time period
q = filter q by 'CreatedDate_Year' >= 2023;

-- Group by week with additional breakdowns
q = group q by (
    'CreatedDate_Year',
    'CreatedDate_Week',
```

```

    'Status',
    'Priority',
    'Origin'
);

q = foreach q generate
    'CreateDate_Year' as 'Year',
    'CreateDate_Week' as 'Week',
    'Status',
    'Priority',
    'Origin',
    count() as 'CaseCount',
    'CreateDate_Year' + "-W" + 'CreateDate_Week' as 'YearWeek',
    date('CreateDate_Year' + "-01-01") + ('CreateDate_Week' * 7) as 'WeekStartDate';

-- Sort by year and week
q = order q by ('Year' asc, 'Week' asc);

```

1.3.2.2 2.2 Enhanced Query with Additional Dimensions

1.3.3 Step 3: Create Dashboard Components

1.3.3.1 3.1 Main Trending Line Chart

1. Create New Dashboard

- Go to **Analytics Studio** → **Create** → **Dashboard**
- Name: “Weekly Case Trending Report”

2. Add Line Chart Widget

- Click **+** **Widget**
- Select **Chart**
- Choose **Line Chart**
- Configure:
 - **X-axis**: YearWeek or WeekStartDate
 - **Y-axis**: CaseCount
 - **Group by**: Status (optional, for stacked view)

3. Chart Configuration

```

{
  "type": "line",
  "query": "YourQueryName",
  "visualizationParameters": {
    "options": {
      "title": "Weekly Case Count Trend",
      "subtitle": "Cases created per week over time"
    },
    "axes": {
      "x": {
        "label": "Week",
        "type": "date"
      },
      "y": {
        "label": "Number of Cases",
        "type": "linear"
      }
    }
  }
}

```

```
}
}
```

1.3.3.2 3.2 Summary KPI Cards Create KPI cards showing: - Total Cases This Week - Average Weekly Cases - Week-over-Week Change - Peak Week Volume

```
-- Current week cases
currentWeek = load "YourCaseDatasetName";
currentWeek = filter currentWeek by 'CreatedDate_Week' == date_part("week", now())
    and 'CreatedDate_Year' == date_part("year", now());
currentWeekCount = group currentWeek by all;
currentWeekCount = foreach currentWeekCount generate count() as 'CurrentWeekCases';

-- Previous week cases
previousWeek = load "YourCaseDatasetName";
previousWeek = filter previousWeek by 'CreatedDate_Week' == (date_part("week", now()) - 1)
    and 'CreatedDate_Year' == date_part("year", now());
previousWeekCount = group previousWeek by all;
previousWeekCount = foreach previousWeekCount generate count() as 'PreviousWeekCases';
```

1.3.3.3 3.3 Breakdown Charts

1. Cases by Status (Donut Chart)
2. Cases by Priority (Bar Chart)
3. Cases by Origin (Horizontal Bar Chart)

1.3.4 Step 4: Add Interactive Filters

```
-- Add date range selection
q = load "YourCaseDatasetName";
q = filter q by 'CreatedDate_Year' >= {{cell(DateRangeStart).asNumber()}}
    and 'CreatedDate_Year' <= {{cell(DateRangeEnd).asNumber()}};
```

1.3.4.1 4.1 Date Range Filter

1.3.4.2 4.2 Multi-Select Filters

- **Status Filter:** Allow selection of specific case statuses
- **Priority Filter:** Filter by case priority levels
- **Origin Filter:** Filter by case origin (Web, Phone, Email, etc.)
- **Owner Filter:** Filter by case owner or team

1.3.5 Step 5: Advanced Features

```
q = load "YourCaseDatasetName";
q = filter q by 'CreatedDate_Year' >= 2023;

-- Group by week
q = group q by ('CreatedDate_Year', 'CreatedDate_Week');
q = foreach q generate
    'CreatedDate_Year' as 'Year',
    'CreatedDate_Week' as 'Week',
    count() as 'CaseCount',
    'CreatedDate_Year' + "-W" + 'CreatedDate_Week' as 'YearWeek';
```

```

-- Calculate 4-week moving average
q = order q by ('Year' asc, 'Week' asc);
q = foreach q generate
    'Year',
    'Week',
    'CaseCount',
    'YearWeek',
    avg('CaseCount') over (order by ('Year', 'Week') rows between 3 preceding and current row) as 'MovingAverage'

```

1.3.5.1 5.1 Trend Analysis with Moving Averages

```

q = load "YourCaseDatasetName";

-- Current year data
currentYear = filter q by 'CreatedDate_Year' == date_part("year", now());
currentYear = group currentYear by 'CreatedDate_Week';
currentYear = foreach currentYear generate
    'CreatedDate_Week' as 'Week',
    count() as 'CurrentYearCases';

-- Previous year data
previousYear = filter q by 'CreatedDate_Year' == (date_part("year", now()) - 1);
previousYear = group previousYear by 'CreatedDate_Week';
previousYear = foreach previousYear generate
    'CreatedDate_Week' as 'Week',
    count() as 'PreviousYearCases';

-- Join for comparison
comparison = cogroup currentYear by 'Week', previousYear by 'Week';
comparison = foreach comparison generate
    'Week',
    sum(currentYear.'CurrentYearCases') as 'CurrentYear',
    sum(previousYear.'PreviousYearCases') as 'PreviousYear',
    (sum(currentYear.'CurrentYearCases') - sum(previousYear.'PreviousYearCases')) as 'YoYChange';

```

1.3.5.2 5.2 Comparative Analysis (Year-over-Year)

1.3.6 Step 6: Dashboard Layout and Design

1.3.6.1 6.1 Recommended Layout Structure

```

+-----+
|           Dashboard Header           |
|           Weekly Case Trending Report |
+-----+
| Total Cases | Avg Weekly | WoW Change | Peak Week |
| This Week  | Cases      | %          | Volume    |
+-----+
|           Main Trending Line Chart   |
|           (Full Width)               |
+-----+
| Cases by   | Cases by   | Cases by   | Top 10 Case |
| Status     | Priority    | Origin      | Owners       |
| (Donut)    | (Bar)       | (H-Bar)     | (Table)      |

```

+-----+-----+-----+-----+

1.3.6.2 6.2 Color Scheme and Styling

- Use consistent color palette
- Apply Salesforce Lightning Design System colors
- Ensure accessibility compliance
- Use conditional formatting for trend indicators

1.3.7 Step 7: Testing and Validation

1.3.7.1 7.1 Data Validation Checklist

- ☐ Verify case counts match Salesforce reports
- ☐ Check date ranges are correct
- ☐ Validate filters work properly
- ☐ Test dashboard performance with large datasets
- ☐ Confirm mobile responsiveness

1.3.7.2 7.2 User Acceptance Testing

- ☐ Test with different user profiles
- ☐ Verify permissions and data access
- ☐ Check dashboard loading times
- ☐ Validate export functionality

1.3.8 Step 8: Deployment and Sharing

1.3.8.1 8.1 Dashboard Sharing

1. **Set Sharing Settings**
 - Navigate to dashboard settings
 - Configure sharing with appropriate users/groups
 - Set view/edit permissions
2. **Create Dashboard URL**
 - Generate shareable link
 - Configure embedded dashboard if needed

1.3.8.2 8.2 Schedule and Notifications

- Set up scheduled dashboard emails
- Configure threshold alerts for unusual case volumes
- Create subscription lists for stakeholders

1.4 Best Practices

1.4.1 Performance Optimization

1. **Use Dataset Filters:** Apply filters at dataset level when possible
2. **Limit Date Ranges:** Don't load unnecessary historical data
3. **Optimize Queries:** Use efficient SAQL patterns
4. **Cache Results:** Enable query result caching

1.4.2 Data Quality

1. **Data Validation:** Implement data quality checks
2. **Handle Null Values:** Account for missing data
3. **Time Zone Considerations:** Ensure consistent time zone handling
4. **Data Refresh:** Set appropriate refresh schedules

1.4.3 User Experience

1. **Intuitive Navigation:** Clear filter labels and options
2. **Loading Indicators:** Show progress for long-running queries
3. **Error Handling:** Provide meaningful error messages
4. **Mobile Optimization:** Ensure dashboard works on mobile devices

1.5 Troubleshooting Common Issues

1.5.1 Query Performance Issues

```
-- Use compact date format for better performance
q = foreach q generate
  date_part("year", 'CreatedDate') as 'Year',
  date_part("week", 'CreatedDate') as 'Week',
  count() as 'CaseCount';
```

1.5.2 Date Handling Problems

```
-- Ensure proper date formatting
q = foreach q generate
  date('CreatedDate_Year' + "-01-01") + (('CreatedDate_Week' - 1) * 7) as 'WeekStartDate';
```

1.5.3 Filter Not Working

- Check field names match dataset schema
- Verify filter syntax in SAQL
- Ensure proper data types

1.5.4 Dashboard Not Loading

- Check query syntax for errors
- Verify dataset permissions
- Review dashboard sharing settings

1.6 Maintenance and Updates

1.6.1 Regular Tasks

1. **Monthly Review:** Check data accuracy and completeness
2. **Quarterly Updates:** Review and update filters/dimensions
3. **Annual Review:** Assess dashboard effectiveness and user feedback

1.6.2 Version Control

- Document changes to queries and dashboard
- Maintain backup copies of working versions
- Test changes in sandbox before production deployment

1.7 Additional Resources

1.7.1 Salesforce Documentation

- [CRM Analytics Developer Guide](#)
- [SAQL Reference](#)
- [Dashboard Design Best Practices](#)

1.7.2 Training Resources

- Trailhead: CRM Analytics Basics
- Trailhead: Advanced Analytics
- Salesforce CRM Analytics Specialist Certification

This comprehensive guide provides all the necessary steps to implement a professional weekly case count trending report in Salesforce CRM Analytics. Follow each step carefully and customize the queries and visualizations based on your specific business requirements.